On Combiners for Verifiable Random Functions

Basil Rohner (brohner@ethz.ch)
Bachelor's Thesis, ETH Zürich
May 7, 2026

Cryptographic Hardness Assumptions

Most cryptographic constructions are built on a *hardness assumption*.

Cryptographic Hardness Assumptions

Most cryptographic constructions are built on a *hardness assumption*.

- Factoring, Diffie–Hellman, LWE, isogenies, …

Cryptographic Hardness Assumptions

Most cryptographic constructions are built on a *hardness assumption*.

- Factoring, Diffie–Hellman, LWE, isogenies, …
- Each one *might* fail: cryptanalysis, quantum computers, …

Cryptographic Hardness Assumptions

Most cryptographic constructions are built on a *hardness assumption*.

- Factoring, Diffie–Hellman, LWE, isogenies, …
- Each one *might* fail: cryptanalysis, quantum computers, …
- A broken assumption breaks every construction that relies on it

Cryptographic Hardness Assumptions

Most cryptographic constructions are built on a *hardness assumption*.

- Factoring, Diffie–Hellman, LWE, isogenies, …
- Each one *might* fail: cryptanalysis, quantum computers, …
- A broken assumption breaks every construction that relies on it

Don’t bet on a single assumption.
Combine candidates based on different ones.

Research Question

Can we *combine* many verifiable random function (VRF) instances such that the resulting VRF is secure if *enough* candidates are secure?

Research Question

Can we *combine* verifiable random function (VRF) instances such that the resulting VRF is secure if *k*-out-of-*n* candidates are secure?

Research Question

Can we *combine* verifiable random function (VRF) instances in a black-box fashion such that the resulting VRF is secure if *k*-out-of-*n* candidates are secure?

Verifiable Functions

Definition (Verifiable Function)

A *verifiable function* (VF) VF is a triple of the following efficient algorithms:

- $\mathsf{Gen}_{\mathsf{VF}}(1^\lambda) = (\mathsf{sk}, \mathsf{pk})$ generates a key-pair
- $\mathsf{Eval}_{\mathsf{VF}}(\mathsf{sk}, x) = (y, \pi)$ evaluates points to an image-proof pair
- $\mathsf{Vfy}_{\mathsf{VF}}(\mathsf{pk}, x, y, \pi) = b$ verifies evaluations

Verifiable Functions

Definition (Verifiable Function)

A *verifiable function* (VF) VF is a triple of the following efficient algorithms:

- $\mathsf{Gen}_{\mathsf{VF}}(1^\lambda) = (\mathsf{sk}, \mathsf{pk})$ generates a key-pair
- $\mathsf{Eval}_{\mathsf{VF}}(\mathsf{sk}, x) = (y, \pi)$ evaluates points to an image-proof pair
- $\mathsf{Vfy}_{\mathsf{VF}}(\mathsf{pk}, x, y, \pi) = b$ verifies evaluations

A VF is a purely syntactic object, we define security properties on top of it.

Properties of Verifiable Functions

ε-Correctness

Honestly generated image-proof pairs verify with probability at least ε:

$$\Pr\left[\begin{array}{l}(\mathsf{sk},\mathsf{pk}) \leftarrow_{\$} \mathsf{Gen}_{\mathsf{VF}}(1^{\lambda}) \\ \quad x \leftarrow_{\$} \{0,1\}^{\lambda} \quad : \mathsf{Vfy}_{\mathsf{VF}}(\mathsf{pk},x,y,\pi) = 1 \\ (y,\pi) \leftarrow \mathsf{Eval}_{\mathsf{VF}}(\mathsf{sk},x)\end{array}\right] \geq \varepsilon(\lambda)$$

VF is *correct* if it is 1-correct and *statistically correct* if it is $(1-\varepsilon)$-correct and $\varepsilon \in \mathsf{negl}(\lambda)$.

Properties of Verifiable Functions

Unique Provability

For every public key pk and every point x, at most one image y is provable:

$$\mathrm{Vfy}_{\mathrm{VRF}}(\mathsf{pk}, x, y_0, \pi_0) = \mathrm{Vfy}_{\mathrm{VRF}}(\mathsf{pk}, x, y_1, \pi_1) = 1 \implies y_0 = y_1.$$

Properties of Verifiable Functions

(ε, t)-Pseudorandomness

For any adversary $\mathcal{A}$ running in time at most t:

$$\mathbf{Adv}^{\mathrm{prf}}_{\mathrm{VF}}(\lambda, \mathcal{A}) := 2 \cdot \left| \Pr\left[\mathbf{Exp}^{\mathrm{prf}}_{\mathrm{VF}}(\lambda, \mathcal{A}) = 1\right] - \frac{1}{2} \right| \leq \varepsilon(\lambda)$$

Pseudorandomness Experiment

Experiment $\mathbf{Exp}^{\mathsf{prf}}_{\mathsf{VF}}(\lambda, \mathcal{A})$

1 : $b \leftarrow_{\$} \{0,1\}$
2 : $(\mathsf{sk}, \mathsf{pk}) \leftarrow_{\$} \mathsf{Gen}_{\mathsf{VF}}(1^{\lambda})$
3 : $S \leftarrow \varnothing$
4 : $(x^*, \sigma) \leftarrow_{\$} \mathcal{A}_1^{\mathcal{O}_{\mathsf{eval}}}(1^{\lambda}, \mathsf{pk})$
5 : $(y_0, \pi_0) \leftarrow \mathsf{Eval}_{\mathsf{VF}}(\mathsf{sk}, x^*)$
6 : $y_1 \leftarrow_{\$} \mathcal{Y}$
7 : $b' \leftarrow_{\$} \mathcal{A}_2(1^{\lambda}, x^*, \sigma, y_b)$
8 : **return** $b = b' \wedge x^* \notin S$

Oracle $\mathcal{O}_{\mathsf{eval}}(x)$

1 : $S \leftarrow S \cup \{x\}$
2 : $(y, \pi) \leftarrow_{\$} \mathsf{Eval}_{\mathsf{VF}}(\mathsf{sk}, x)$
3 : **return** y

Properties of Verifiable Functions

(ε, t)-Weak Pseudorandomness

For any adversary $\mathcal{A}$ running in time at most t in the wprf game:

$$\mathbf{Adv}^{\mathrm{wprf}}_{\mathrm{VF}}(\lambda, \mathcal{A}) := 2 \cdot \left| \Pr\left[\mathbf{Exp}^{\mathrm{wprf}}_{\mathrm{VF}}(\lambda, \mathcal{A}) = 1\right] - \frac{1}{2} \right| \leq \varepsilon(\lambda)$$

Weakly pseudorandom if $\varepsilon \in \mathsf{negl}(\lambda)$ for $t \in \mathsf{poly}(\lambda)$.

Weak Pseudorandomness Experiment

Experiment $\mathbf{Exp}^{\mathsf{wprf}}_{\mathsf{VF}}(\lambda, \mathcal{A})$

1 : $b \leftarrow_{\$} \{0,1\}$
2 : $(\mathsf{sk}, \mathsf{pk}) \leftarrow_{\$} \mathsf{Gen}_{\mathsf{VF}}(1^{\lambda})$
3 : $S \leftarrow \varnothing$
4 : $\sigma \leftarrow_{\$} \mathcal{A}_1^{\mathcal{O}_{\mathsf{eval}}}(1^{\lambda}, \mathsf{pk})$
5 : $x^* \leftarrow_{\$} \{0,1\}^{\lambda}$
6 : $(y_0, \pi_0) \leftarrow \mathsf{Eval}_{\mathsf{VF}}(\mathsf{sk}, x^*)$
7 : $y_1 \leftarrow_{\$} \mathcal{Y}$
8 : $b' \leftarrow_{\$} \mathcal{A}_2(1^{\lambda}, x^*, \sigma, y_b)$
9 : **return** $b = b' \wedge x^* \notin S$

Oracle $\mathcal{O}_{\mathsf{eval}}(x)$

1 : $S \leftarrow S \cup \{x\}$
2 : $(y, \pi) \leftarrow_{\$} \mathsf{Eval}_{\mathsf{VF}}(\mathsf{sk}, x)$
3 : **return** y

Properties of Verifiable Functions

(ε, t)-Unpredictability

For any adversary $\mathcal{A}$ running in time at most t in the up game:

$$\mathbf{Adv}^{\mathsf{up}}_{\mathsf{VF}}(\lambda, \mathcal{A}) := \Pr\left[\mathbf{Exp}^{\mathsf{up}}_{\mathsf{VF}}(\lambda, \mathcal{A}) = 1\right] \leq \varepsilon(\lambda)$$

Unpredictable if $\varepsilon \in \mathsf{negl}(\lambda)$ for $t \in \mathsf{poly}(\lambda)$.

Unpredictability Experiment

Experiment $\mathbf{Exp}^{\mathsf{up}}_{\mathsf{VF}}(\lambda, \mathcal{A})$

1 : $(\mathsf{sk}, \mathsf{pk}) \leftarrow_{\$} \mathsf{Gen}_{\mathsf{VF}}(1^{\lambda})$
2 : $S \leftarrow \varnothing$
3 : $(x^*, y^*) \leftarrow_{\$} \mathcal{A}^{\mathcal{O}_{\mathsf{eval}}}(1^{\lambda}, \mathsf{pk})$
4 : $(y, \pi) \leftarrow \mathsf{Eval}_{\mathsf{VF}}(\mathsf{sk}, x^*)$
5 : **return** $y^* = y \wedge x^* \notin S$

Oracle $\mathcal{O}_{\mathsf{eval}}(x)$

1 : $S \leftarrow S \cup \{x\}$
2 : $(y, \pi) \leftarrow_{\$} \mathsf{Eval}_{\mathsf{VF}}(\mathsf{sk}, x)$
3 : **return** y

Verifiable Random Functions

Verifiable Random Function

A *verifiable random function* (VRF) is a VF that is:

- Correct
- Uniquely Provable
- Pseudorandom

Weak Verifiable Random Functions

Weak Verifiable Random Function

A *weak verifiable random function* (wVRF) is a VF that is:

- Correct
- Uniquely Provable
- Weakly Pseudorandom

Verifiable Unpredictable Functions

Verifiable Unpredictable Function

A *verifiable unpredictable function* (VUF) is a VF that is:

- Correct
- Uniquely Provable
- Unpredictable

Why Should We Care About VRFs?

- Signatures with Strong Verifiability Guarantees [GO93]

Why Should We Care About VRFs?

- Signatures with Strong Verifiability Guarantees [GO93]
- Resettable Zero-Knowledge proofs [MR01]

Why Should We Care About VRFs?

- Signatures with Strong Verifiability Guarantees [GO93]
- Resettable Zero-Knowledge proofs [MR01]
- Distributed Leader Election in PoS blockchains [KRDO17; DGKR18; BASV23]

Robust Black-Box Combiners

Definition (Robust $(n; k)$ Black-Box Combiner (Informal))

Robust $(n; k)$ black-box P-combiner Π_P satisfy:

- Give oracle access to n candidate schemes $\mathcal{O}_1, \ldots, \mathcal{O}_n$
- Implements primitive P
- Is secure if at least k candidates are secure

Robust Black-Box Combiners

Definition (Robust $(n; k)$ Black-Box Combiner (Informal))

Robust $(n; k)$ black-box P-combiner Π_P satisfy:

- Give oracle access to n candidate schemes $\mathcal{O}_1, \ldots, \mathcal{O}_n$
- Implements primitive P
- Is secure if at least k candidates are secure **even against inefficient adversaries**

Robust Black-Box Combiners

Definition (Robust $(n; k)$ Black-Box Combiner (Informal))

Robust $(n; k)$ black-box P-combiner Π_P satisfy:

- Give oracle access to n candidate schemes $\mathcal{O}_1, \ldots, \mathcal{O}_n$
- Implements primitive P
- Is secure if at least k candidates are secure **even against adversaries with a PSPACE oracle**

Example

Algorithm $\Pi_{\mathsf{PRF}}^{\mathcal{O}_1,\mathcal{O}_2}.\mathsf{Eval}_{\mathsf{PRF}}(k_1, k_2, x)$

1 : $y_1 \leftarrow \mathcal{O}_1.\mathsf{Eval}_{\mathsf{PRF}}(k_1, x)$
2 : $y_2 \leftarrow \mathcal{O}_2.\mathsf{Eval}_{\mathsf{PRF}}(k_2, x)$
3 : **return** $y_1 \oplus y_2$

Figure: Robust $(2; 1)$ Black-Box PRF-Combiner.

Robust Black-Box DAG Combiners

Definition (Robust $(n; k)$ Black-Box DAG Combiner, informal)

Each algorithm of $\widetilde{\Pi}_P$ is given by a directed acyclic graph $\mathcal{G}$:

- source s, sink t, intermediate vertices labeled $\ell : V(\mathcal{G}) \to [n] \times [|P|]$
- at each vertex v, a PPT machine f^v computes a query $\mathbf{q}_v$ from predecessors' outputs and queries $\mathcal{O}_{\ell(v)}$

Execute in topological order. The sink emits the final output.

Robust Black-Box DAG Combiners

Definition (Robust $(n; k)$ Black-Box DAG Combiner, informal)

Each algorithm of $\widetilde{\Pi}_P$ is given by a directed acyclic graph $\mathcal{G}$:

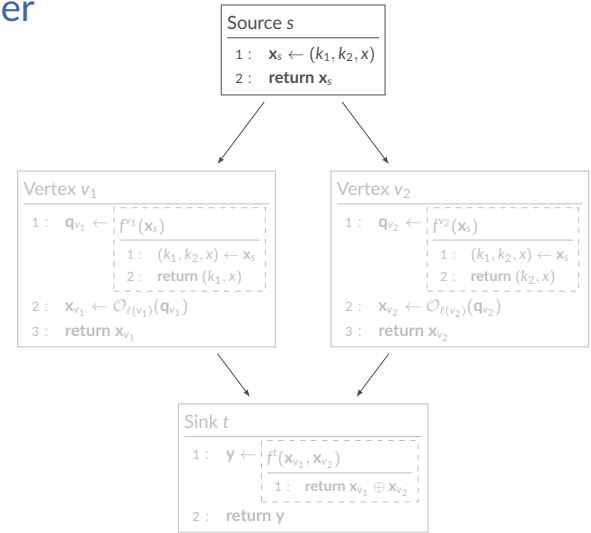
- source s, sink t, intermediate vertices labeled $\ell : V(\mathcal{G}) \to [n] \times [|P|]$
- at each vertex v, a PPT machine f^v computes a query $\mathbf{q}_v$ from predecessors' outputs and queries $\mathcal{O}_{\ell(v)}$

Execute in topological order. The sink emits the final output.

Why DAGs? Decouples local computation from oracle queries. Structure becomes explicit.

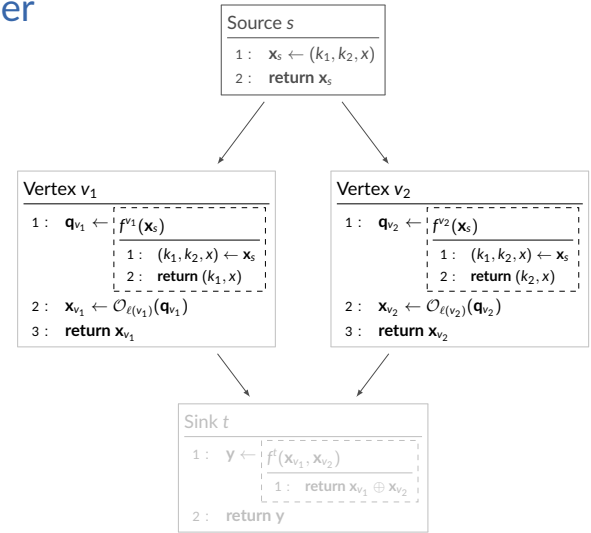
Example: DAG PRF-Combiner

- *Source s*: forwards (k_1, k_2, x).



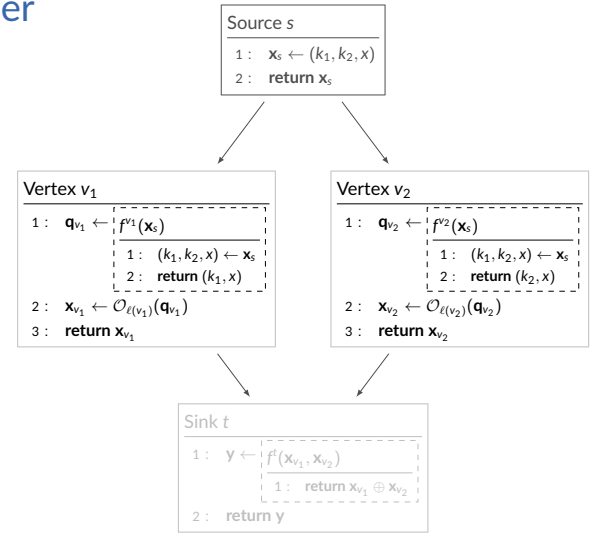
Example: DAG PRF-Combiner

- *Source s*: forwards (k_1, k_2, x).
- *Intermediate vertices*:
  - PPT TM f^v pre-computes the query



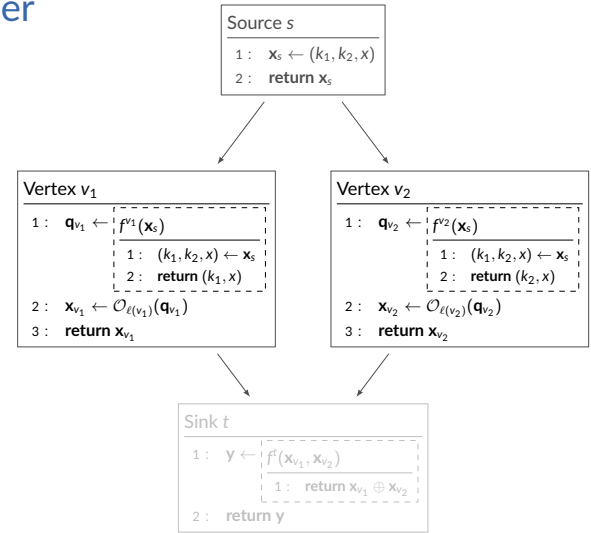
Example: DAG PRF-Combiner

- *Source s*: forwards (k_1, k_2, x).
- *Intermediate vertices*:
  - PPT TM f^v pre-computes the query
  - query the candidate oracle



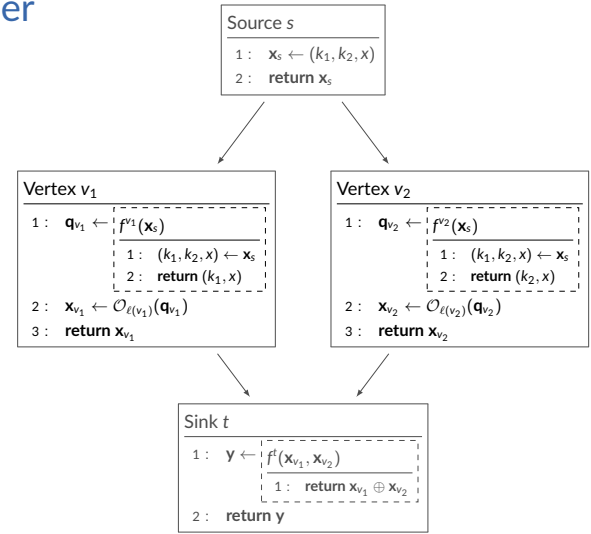
Example: DAG PRF-Combiner

- *Source s*: forwards (k_1, k_2, x).
- *Intermediate vertices*:
  - PPT TM f^v pre-computes the query
  - query the candidate oracle
  - return the response



Example: DAG PRF-Combiner

- *Source s*: forwards (k_1, k_2, x).
- *Intermediate vertices*:
  - PPT TM f^v pre-computes the query
  - query the candidate oracle
  - return the response
- *Sink t*: computes the final output.



Equivalence

- Robust $(n; k)$ black-box DAG combiner $\implies$ robust $(n; k)$ black-box combiner

Equivalence

- Robust $(n; k)$ black-box DAG combiner $\implies$ robust $(n; k)$ black-box combiner
- Robust $(n; k)$ black-box combiner $\implies$ robust $(n; k)$ black-box DAG combiner

Equivalence

- Robust $(n; k)$ black-box DAG combiner $\implies$ robust $(n; k)$ black-box combiner
- Robust $(n; k)$ black-box combiner $\implies$ robust $(n; k)$ black-box DAG combiner

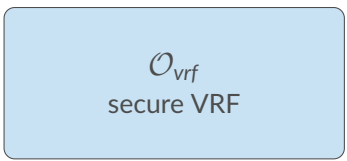
It suffices to (dis)prove the DAG version.

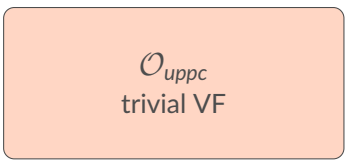
Main Theorem

There exists no robust $(2; 1)$ black-box VRF-combiner.

The Two Candidates

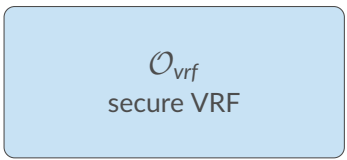
Suppose $\widetilde{\Pi}_{\mathsf{VRF}}$ is a robust $(2; 1)$ black-box DAG VRF-combiner.

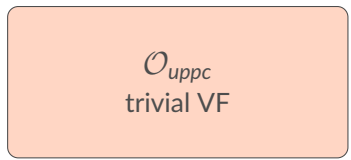




The Two Candidates

Suppose $\widetilde{\Pi}_{\mathsf{VRF}}$ is a robust $(2;1)$ black-box DAG VRF-combiner.

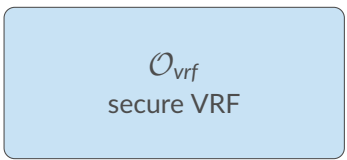


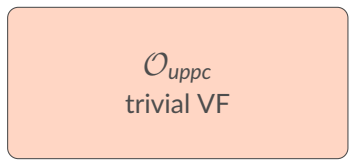


$\mathcal{O}_{uppc}$: every $\mathsf{Eval}(\mathsf{sk}, x)$ returns the same default y_{def}.

The Two Candidates

Suppose $\widetilde{\Pi}_{\mathsf{VRF}}$ is a robust $(2;1)$ black-box DAG VRF-combiner.



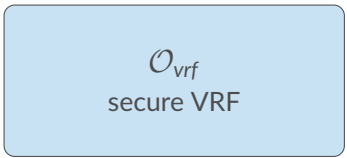


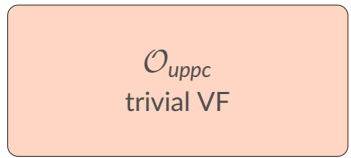
$\mathcal{O}_{uppc}$: every $\mathsf{Eval}(\mathsf{sk}, x)$ returns the same default y_{def}.

$\mathcal{O}_{vrf}$ secure $\Rightarrow$ $\widetilde{\Pi}_{\mathsf{VRF}}^{\mathcal{O}_{vrf},\mathcal{O}_{uppc}}$ is a secure VRF.

The Two Candidates

Suppose $\widetilde{\Pi}_{\mathrm{VRF}}$ is a robust $(2;1)$ black-box DAG VRF-combiner.





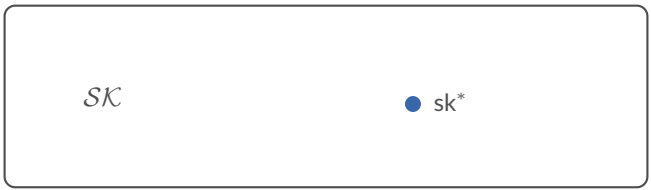
$\mathcal{O}_{uppc}$: every $\mathrm{Eval}(\mathrm{sk}, x)$ returns the same default y_{def}.

$\mathcal{O}_{vrf}$ secure $\Rightarrow \widetilde{\Pi}_{\mathrm{VRF}}^{\mathcal{O}_{vrf},\mathcal{O}_{uppc}}$ is a secure VRF.

Goal: adversary $\mathcal{A}^{\mathrm{PSPACE}}$ breaking its pseudorandomness.

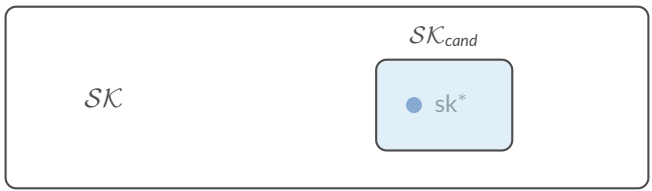
The Idea

The pseudorandomness experiment hides a secret key $\mathsf{sk}^* \in \mathcal{SK} = \{0,1\}^{s(\lambda)}$.



The Idea

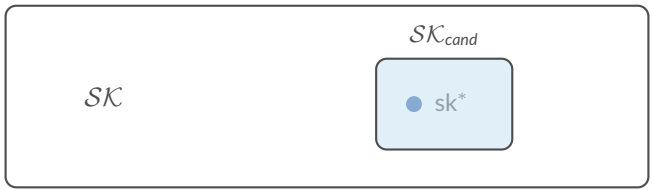
The pseudorandomness experiment hides a secret key $\mathsf{sk}^* \in \mathcal{SK} = \{0,1\}^{s(\lambda)}$.



Use the PSPACE oracle to *filter* $\mathcal{SK}$ down to a small $\mathcal{SK}_{cand}$ that still contains sk^*.

The Idea

The pseudorandomness experiment hides a secret key $\mathsf{sk}^* \in \mathcal{SK} = \{0,1\}^{s(\lambda)}$.



Use the PSPACE oracle to *filter* $\mathcal{SK}$ down to a small $\mathcal{SK}_{cand}$ that still contains sk^*.
If $|\mathcal{SK}_{cand}| \in \mathsf{poly}$, sampling uniformly gives $\mathsf{sk}_{cand} = \mathsf{sk}^*$ with non-negligible probability.

$\mathsf{EvalSim}_{\mathsf{VRF}}$ as a Black Box

Assume an algorithm $\mathsf{EvalSim}_{\mathsf{VRF}}(\mathsf{sk}, x, y, \pi, \tau) \in \{\top, \bot\}$ with the following properties:

$\mathsf{EvalSim}_{\mathsf{VRF}}$ as a Black Box

Assume an algorithm $\mathsf{EvalSim}_{\mathsf{VRF}}(\mathsf{sk}, x, y, \pi, \tau) \in \{\top, \bot\}$ with the following properties:

- makes no oracle queries

$\mathsf{EvalSim}_{\mathsf{VRF}}$ as a Black Box

Assume an algorithm $\mathsf{EvalSim}_{\mathsf{VRF}}(\mathsf{sk}, x, y, \pi, \tau) \in \{\top, \bot\}$ with the following properties:

- makes no oracle queries
- returns $\top$ on the experiment's secret key sk^*

$\mathsf{EvalSim}_{\mathsf{VRF}}$ as a Black Box

Assume an algorithm $\mathsf{EvalSim}_{\mathsf{VRF}}(\mathsf{sk}, x, y, \pi, \tau) \in \{\top, \bot\}$ with the following properties:

- makes no oracle queries
- returns $\top$ on the experiment's secret key sk^*
- returns $\bot$ when sk's combiner output disagrees with y

$\mathsf{EvalSim}_{\mathsf{VRF}}$ as a Black Box

Assume an algorithm $\mathsf{EvalSim}_{\mathsf{VRF}}(\mathsf{sk}, x, y, \pi, \tau) \in \{\top, \bot\}$ with the following properties:

- makes no oracle queries
- returns $\top$ on the experiment's secret key sk^*
- returns $\bot$ when sk's combiner output disagrees with y
- runs in polynomial space

$\mathsf{EvalSim}_{\mathsf{VRF}}$ as a Black Box

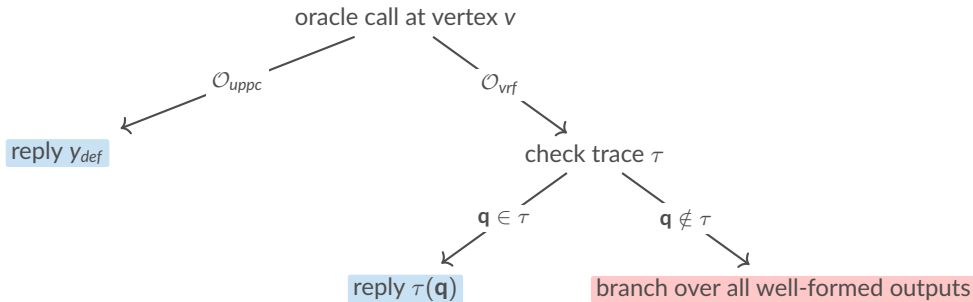
Assume an algorithm $\mathsf{EvalSim}_{\mathsf{VRF}}(\mathsf{sk}, x, y, \pi, \tau) \in \{\top, \bot\}$ with the following properties:

- makes no oracle queries
- returns $\top$ on the experiment's secret key sk^*
- returns $\bot$ when sk's combiner output disagrees with y
- runs in polynomial space

Constructing $\mathsf{EvalSim}_{\mathsf{VRF}}$ *and proving these properties hold is the technical core of the thesis.*

Constructing $\mathsf{EvalSim}_{\mathsf{VRF}}$

Traverse the DAG in topological order. At each oracle-call vertex:



Return $\top$ iff *every* branch produces the honest image y.

$\mathsf{EvalSim}_{\mathsf{VRF}}$ Revisited

Properties of $\mathsf{EvalSim}_{\mathsf{VRF}}(\mathsf{sk}, x, y, \pi, \tau)$:

- makes no oracle queries
- returns $\top$ on the experiment's secret key sk^*
- returns $\bot$ when sk's combiner output disagrees with y
- runs in polynomial space

$\mathsf{EvalSim}_{\mathsf{VRF}}$ Revisited

Properties of $\mathsf{EvalSim}_{\mathsf{VRF}}(\mathsf{sk}, x, y, \pi, \tau)$:

- ✓ makes no oracle queries
- returns $\top$ on the experiment's secret key sk^*
- returns $\bot$ when sk's combiner output disagrees with y
- runs in polynomial space

$\mathsf{EvalSim}_{\mathsf{VRF}}$ Revisited

Properties of $\mathsf{EvalSim}_{\mathsf{VRF}}(\mathsf{sk}, x, y, \pi, \tau)$:

- ✓ makes no oracle queries
- • returns $\top$ on the experiment's secret key sk^*
- ✓ returns $\bot$ when sk's combiner output disagrees with y
- • runs in polynomial space

$\mathsf{EvalSim}_{\mathsf{VRF}}$ Revisited

Properties of $\mathsf{EvalSim}_{\mathsf{VRF}}(\mathsf{sk}, x, y, \pi, \tau)$:

- ✓ makes no oracle queries
- • returns $\top$ on the experiment's secret key sk^*
- ✓ returns $\bot$ when sk's combiner output disagrees with y
- ✓ runs in polynomial space

The Adversary

1. Sample $x_1, \ldots, x_{s+\lambda} \leftarrow_\$ \{0,1\}^\lambda$ (recall $\mathcal{SK} = \{0,1\}^{s(\lambda)}$).

The Adversary

1. Sample $x_1, \ldots, x_{s+\lambda} \leftarrow_\$ \{0,1\}^\lambda$ (recall $\mathcal{SK} = \{0,1\}^{s(\lambda)}$).
2. Query the eval oracle: $(y_i, \pi_i) \leftarrow \mathcal{O}_{\mathsf{eval}}(x_i)$.

The Adversary

1. Sample $x_1, \ldots, x_{s+\lambda} \leftarrow_{\$} \{0,1\}^{\lambda}$ (recall $\mathcal{SK} = \{0,1\}^{s(\lambda)}$).
2. Query the eval oracle: $(y_i, \pi_i) \leftarrow \mathcal{O}_{\mathsf{eval}}(x_i)$.
3. Compute the verification traces $\tau_i \leftarrow \mathsf{trace}\big(\widetilde{\Pi}_{\mathsf{VRF}}^{\mathcal{O}_{vrf}, \mathcal{O}_{uppc}}.\mathsf{Vfy}(\mathsf{pk}^*, x_i, y_i, \pi_i)\big)$.

The Adversary

1. Sample $x_1, \ldots, x_{s+\lambda} \leftarrow_\$ \{0,1\}^\lambda$ (recall $\mathcal{SK} = \{0,1\}^{s(\lambda)}$).
2. Query the eval oracle: $(y_i, \pi_i) \leftarrow \mathcal{O}_{\mathsf{eval}}(x_i)$.
3. Compute the verification traces $\tau_i \leftarrow \mathrm{trace}\big(\widetilde{\Pi}_{\mathsf{VRF}}^{\mathcal{O}_{vrf}, \mathcal{O}_{uppc}}.\mathsf{Vfy}(\mathsf{pk}^*, x_i, y_i, \pi_i)\big)$.
4. Use PSPACE to sample $\mathsf{sk}_{cand} \leftarrow_\$ \mathcal{SK}_{cand}$, where

$$\mathcal{SK}_{cand} := \big\{\mathsf{sk} \in \mathcal{SK} : \mathsf{EvalSim}_{\mathsf{VRF}}(\mathsf{sk}, x_i, y_i, \pi_i, \tau_i) = \top \ \forall i\big\}.$$

The Adversary

1. Sample $x_1, \ldots, x_{s+\lambda} \leftarrow_\$ \{0,1\}^\lambda$ (recall $\mathcal{SK} = \{0,1\}^{s(\lambda)}$).
2. Query the eval oracle: $(y_i, \pi_i) \leftarrow \mathcal{O}_{\mathsf{eval}}(x_i)$.
3. Compute the verification traces $\tau_i \leftarrow \mathsf{trace}\big(\widetilde{\Pi}_{\mathsf{VRF}}^{\mathcal{O}_{vrf}, \mathcal{O}_{uppc}}.\mathsf{Vfy}(\mathsf{pk}^*, x_i, y_i, \pi_i)\big)$.
4. Use PSPACE to sample $\mathsf{sk}_{cand} \leftarrow_\$ \mathcal{SK}_{cand}$, where
$$\mathcal{SK}_{cand} := \big\{\mathsf{sk} \in \mathcal{SK} : \mathsf{EvalSim}_{\mathsf{VRF}}(\mathsf{sk}, x_i, y_i, \pi_i, \tau_i) = \top \ \forall i\big\}.$$
5. Sample challenge $x^* \leftarrow_\$ \{0,1\}^\lambda$ and obtain y^*.

The Adversary

1. Sample $x_1, \ldots, x_{s+\lambda} \leftarrow_\$ \{0,1\}^\lambda$ (recall $\mathcal{SK} = \{0,1\}^{s(\lambda)}$).
2. Query the eval oracle: $(y_i, \pi_i) \leftarrow \mathcal{O}_{\mathsf{eval}}(x_i)$.
3. Compute the verification traces $\tau_i \leftarrow \mathrm{trace}\big(\widetilde{\Pi}_{\mathsf{VRF}}^{\mathcal{O}_{vrf},\mathcal{O}_{uppc}}.\mathsf{Vfy}(\mathsf{pk}^*, x_i, y_i, \pi_i)\big)$.
4. Use PSPACE to sample $\mathsf{sk}_{cand} \leftarrow_\$ \mathcal{SK}_{cand}$, where
$$\mathcal{SK}_{cand} := \big\{\mathsf{sk} \in \mathcal{SK} : \mathsf{EvalSim}_{\mathsf{VRF}}(\mathsf{sk}, x_i, y_i, \pi_i, \tau_i) = \top \ \forall i\big\}.$$
5. Sample challenge $x^* \leftarrow_\$ \{0,1\}^\lambda$ and obtain y^*.
6. Check $\widetilde{\Pi}_{\mathsf{VRF}}^{\mathcal{O}_{vrf},\mathcal{O}_{uppc}}.\mathsf{Eval}(\mathsf{sk}_{cand}, x^*) \stackrel{?}{=} (y^*, \pi)$.

Bounding the Advantage

$$\mathbf{Adv}^{\mathsf{prf}}_{\widetilde{\Pi}^{\mathcal{O}_{vrf},\mathcal{O}_{uppc}}_{\mathsf{VRF}}}(\lambda,\mathcal{A}) = 2 \cdot \left|\Pr[\mathcal{A}\text{ wins}] - \tfrac{1}{2}\right|$$

Bounding the Advantage

$$\mathbf{Adv}^{\mathsf{prf}}_{\widetilde{\Pi}_{\mathsf{VRF}}^{\mathcal{O}_{vrf},\mathcal{O}_{uppc}}}(\lambda,\mathcal{A}) = 2\cdot\left|\Pr[\mathcal{A}\text{ wins}]-\tfrac{1}{2}\right|$$

$$\geq 2\cdot\left|\Pr[\mathsf{sk}_{cand}\text{ and }\mathsf{sk}^{*}\text{ produce the same eval image on fresh }x^{*}]-\tfrac{1}{2}\right|$$

Bounding the Advantage

$$\begin{aligned}
\mathbf{Adv}^{\mathrm{prf}}_{\widetilde{\Pi}_{\mathrm{VRF}}^{\mathcal{O}_{vrf},\mathcal{O}_{uppc}}}(\lambda,\mathcal{A}) &= 2\cdot\left|\Pr[\mathcal{A}\text{ wins}]-\tfrac{1}{2}\right| \\
&\geq 2\cdot\left|\Pr[\mathsf{sk}_{cand}\text{ and }\mathsf{sk}^{*}\text{ produce the same eval image on fresh }x^{*}]-\tfrac{1}{2}\right| \\
&\geq \frac{1}{2}-\frac{1}{2^{\lambda+1}}-\frac{1}{|\mathcal{Y}|} \quad \text{(by a Markov argument)}
\end{aligned}$$

Bounding the Advantage

$$\begin{aligned}
\mathbf{Adv}^{\mathsf{prf}}_{\widetilde{\Pi}^{\mathcal{O}_{vrf},\mathcal{O}_{uppc}}_{\mathsf{VRF}}}(\lambda,\mathcal{A}) &= 2\cdot\left|\Pr[\mathcal{A}\text{ wins}]-\tfrac{1}{2}\right| \\
&\geq 2\cdot\left|\Pr[\mathsf{sk}_{cand}\text{ and }\mathsf{sk}^{*}\text{ produce the same eval image on fresh }x^{*}]-\tfrac{1}{2}\right| \\
&\geq \frac{1}{2}-\frac{1}{2^{\lambda+1}}-\frac{1}{|\mathcal{Y}|} \quad \text{(by a Markov argument)}
\end{aligned}$$

$$\mathbf{Adv}^{\mathsf{prf}}_{\widetilde{\Pi}^{\mathcal{O}_{vrf},\mathcal{O}_{uppc}}_{\mathsf{VRF}}}(\lambda,\mathcal{A}) \notin \mathsf{negl}(\lambda) \;\Rightarrow\; \text{contradicts pseudorandomness.} \quad \square$$

Results

Theorem

No robust $(2; 1)$ black-box VRF-combiner.

Corollary

No robust $(n; \lfloor n/2 \rfloor)$ black-box VRF-combiner.

Conjecture

No robust $(n; n - 1)$ black-box VRF-combiner.

Results

Theorem

No robust $(2; 1)$ black-box VRF/VUF/wVRF-combiner.

Corollary

No robust $(n; \lfloor n/2 \rfloor)$ black-box VRF/VUF/wVRF-combiner.

Conjecture

No robust $(n; n - 1)$ black-box VRF/VUF/wVRF-combiner.

Extending to $(n; \lfloor n/2 \rfloor)$ Combiners

Fill the n slots of an $(n; \lfloor n/2 \rfloor)$ combiner with two oracles $\mathcal{O}_1, \mathcal{O}_2$.

Extending to $(n; \lfloor n/2 \rfloor)$ Combiners

Fill the n slots of an $(n; \lfloor n/2 \rfloor)$ combiner with two oracles $\mathcal{O}_1, \mathcal{O}_2$.

Security of either $\Rightarrow \geq \lfloor n/2 \rfloor$ slots secure $\Rightarrow$ a $(2; 1)$ combiner.

Extending to $(n; \lfloor n/2 \rfloor)$ Combiners

Fill the n slots of an $(n; \lfloor n/2 \rfloor)$ combiner with two oracles $\mathcal{O}_1, \mathcal{O}_2$.

Security of either $\Rightarrow \geq \lfloor n/2 \rfloor$ slots secure $\Rightarrow$ a $(2; 1)$ combiner.

Contraposition $\Rightarrow$ no $(n; \lfloor n/2 \rfloor)$ black-box VRF-combiner.

Beyond Impossibility

If all candidates are *uniquely provable* and *perfectly correct* (information-theoretic, typically proven):

Beyond Impossibility

If all candidates are *uniquely provable* and *perfectly correct* (information-theoretic, typically proven):

a simple robust $(n; 1)$ black-box VRF-combiner exists.

Robust $(n; 1)$ Black-Box UP-VRF-Combiner

$\Pi_{\mathsf{VRF}}^{\mathcal{O}_1,\mathcal{O}_2}.\mathsf{Eval}_{\mathsf{VRF}}(\mathsf{sk}, x)$

1 : $(\mathsf{sk}_1, \mathsf{sk}_2) \leftarrow \mathsf{sk}$
2 : $(y_1, \pi_1) \leftarrow \mathcal{O}_1.\mathsf{Eval}_{\mathsf{VRF}}(\mathsf{sk}_1, x)$
3 : $(y_2, \pi_2) \leftarrow \mathcal{O}_2.\mathsf{Eval}_{\mathsf{VRF}}(\mathsf{sk}_2, x)$
4 : $y \leftarrow y_1 \oplus y_2$
5 : $\pi \leftarrow (y_1, \pi_1, y_2, \pi_2)$
6 : **return** (y, π)

$\Pi_{\mathsf{VRF}}^{\mathcal{O}_1,\mathcal{O}_2}.\mathsf{Vfy}_{\mathsf{VRF}}(\mathsf{pk}, x, y, \pi)$

1 : $(\mathsf{pk}_1, \mathsf{pk}_2) \leftarrow \mathsf{pk}$
2 : $(y_1, \pi_1, y_2, \pi_2) \leftarrow \pi$
3 : $b_1 \leftarrow \mathcal{O}_1.\mathsf{Vfy}_{\mathsf{VRF}}(\mathsf{pk}_1, x, y_1, \pi_1)$
4 : $b_2 \leftarrow \mathcal{O}_2.\mathsf{Vfy}_{\mathsf{VRF}}(\mathsf{pk}_2, x, y_2, \pi_2)$
5 : **return** $b_1 \wedge b_2 \wedge (y = y_1 \oplus y_2)$

Robust $(n; 1)$ Black-Box UP-VRF-Combiner

$\Pi^{\mathcal{O}_1,\mathcal{O}_2}_{\mathsf{VRF}}.\mathsf{Eval}_{\mathsf{VRF}}(\mathsf{sk}, x)$

1 : $(\mathsf{sk}_1, \mathsf{sk}_2) \leftarrow \mathsf{sk}$
2 : $(y_1, \pi_1) \leftarrow \mathcal{O}_1.\mathsf{Eval}_{\mathsf{VRF}}(\mathsf{sk}_1, x)$
3 : $(y_2, \pi_2) \leftarrow \mathcal{O}_2.\mathsf{Eval}_{\mathsf{VRF}}(\mathsf{sk}_2, x)$
4 : $y \leftarrow y_1 \oplus y_2$
5 : $\pi \leftarrow (y_1, \pi_1, y_2, \pi_2)$
6 : **return** (y, π)

$\Pi^{\mathcal{O}_1,\mathcal{O}_2}_{\mathsf{VRF}}.\mathsf{Vfy}_{\mathsf{VRF}}(\mathsf{pk}, x, y, \pi)$

1 : $(\mathsf{pk}_1, \mathsf{pk}_2) \leftarrow \mathsf{pk}$
2 : $(y_1, \pi_1, y_2, \pi_2) \leftarrow \pi$
3 : $b_1 \leftarrow \mathcal{O}_1.\mathsf{Vfy}_{\mathsf{VRF}}(\mathsf{pk}_1, x, y_1, \pi_1)$
4 : $b_2 \leftarrow \mathcal{O}_2.\mathsf{Vfy}_{\mathsf{VRF}}(\mathsf{pk}_2, x, y_2, \pi_2)$
5 : **return** $b_1 \wedge b_2 \wedge (y = y_1 \oplus y_2)$

Iterate pairwise to obtain a robust $(n; 1)$ combiner.

Summary

- Introduced Robust $(n; k)$ Black-Box DAG Combiner for more principled analysis

Summary

- Introduced Robust $(n; k)$ Black-Box DAG Combiner for more principled analysis
- Established impossibility for robust $(n; \lfloor n/2 \rfloor)$ black-box VRF-combiner

Summary

- Introduced Robust $(n; k)$ Black-Box DAG Combiner for more principled analysis
- Established impossibility for robust $(n; \lfloor n/2 \rfloor)$ black-box VRF-combiner
- Conjectured impossibility for robust $(n; n - 1)$ black-box VRF-combiner

Summary

- Introduced Robust $(n; k)$ Black-Box DAG Combiner for more principled analysis
- Established impossibility for robust $(n; \lfloor n/2 \rfloor)$ black-box VRF-combiner
- Conjectured impossibility for robust $(n; n-1)$ black-box VRF-combiner
- Constructed robust $(n; 1)$ black-box VRF-combiner given u.p. and p.c. candidates

References I

[BASV23] J. Burdges, H. K. Alper, A. Stewart, and S. Vasilyev. Sassafras and Semi-Anonymous Single Leader Election. Cryptology ePrint Archive, Report 2023/031, 2023.

[DGKR18] B. David, P. Gaži, A. Kiayias, and A. Russell. Ouroboros Praos: An Adaptively-Secure, Semi-synchronous Proof-of-Stake Blockchain. In *EUROCRYPT 2018, Part II*, volume 10821 of *LNCS*, pages 66–98. Springer, 2018.

[GO93] S. Goldwasser and R. Ostrovsky. Invariant Signatures and Non-Interactive Zero-Knowledge Proofs are Equivalent (Extended Abstract). In *CRYPTO'92*, volume 740 of *LNCS*, pages 228–245. Springer, 1993.

[KRDO17] A. Kiayias, A. Russell, B. David, and R. Oliynykov. Ouroboros: A Provably Secure Proof-of-Stake Blockchain Protocol. In *CRYPTO 2017, Part I*, volume 10401 of *LNCS*, pages 357–388. Springer, 2017.

[MR01] S. Micali and L. Reyzin. Soundness in the Public-Key Model. In *CRYPTO 2001*, volume 2139 of *LNCS*, pages 542–565. Springer, 2001.